

Project 6 (Capstone): Cloud-Native Order Platform on AWS

This is the capstone. Where Projects 1 to 5 taught students to ship one service through a hardened pipeline, this project teaches them to architect, build, secure, and operate a complete system of services on AWS, the way a real engineering team delivers for a client. When it is finished, a student can sit across from a hiring manager and describe a genuine cloud-native platform they built end to end.

It is delivered in two phases so the learning is paced: Phase 1 builds the system and runs it; Phase 2 operates it like a production platform.

Start here: what this project is, in plain words

If you are new to enterprise platform work, read this section slowly. Everything later in the document builds on it.

You are going to build an online store, called ShopFlow, the way a professional team would build it for a paying client. Not one big program, but several small programs that each do one job and talk to each other over the network. That style is called **microservices**, and learning to build, connect, secure, and run them on the cloud is the whole point of this project.

Here is the one-paragraph version you can memorise and say out loud:

"ShopFlow is a cloud-native online store I built on AWS. It is a microservices system: a web storefront, a service for the product catalog, a service for placing orders, and a worker that sends order confirmations. The services talk to each other over the network, store data in a managed database, and run on Kubernetes. Everything is shipped automatically through a CI/CD pipeline with testing and security checks, and it is monitored in production."

By the end you will understand every word of that sentence, because you will have built the thing it describes.

The technologies, in plain language

You will meet a lot of names. Here is what each one is and why it is there, in one or two sentences. Do not memorise these yet; just get the gist, and they will become real as you build.

Spring Boot (this is the big one)

Spring Boot is the standard framework for building backend services in Java. A "backend service" is a program with no screen of its own; it waits for requests over the network, does some work (look up a product, save an order), and sends an answer back, usually as JSON. Spring Boot exists so you do not have to write all the plumbing for that yourself. It handles the web server, turning incoming requests into Java method calls, talking to the database, validating input, and more, so you write mostly

the logic that matters. It is what most large companies use for Java backends, which is exactly why you are learning it. One feature worth naming now: Spring Boot **Actuator** adds ready-made health and metrics endpoints to every service, which is how Kubernetes and your monitoring later know whether a service is healthy.

When a hiring manager asks "what is Spring Boot?", you can say: it is the most common Java framework for building REST APIs and backend services, and it removes most of the boilerplate so you focus on business logic.

The rest of the stack, one line each

- **REST API:** the common way services talk over the web. One service sends a request to a web address (like `/products/42`) and gets back data as JSON.
- **Microservice:** one small backend service that does one job and can be built, deployed, and scaled on its own.
- **React:** a popular tool for building the web page the user sees in their browser (the frontend).
- **Container (Docker image):** your service packaged with everything it needs to run, so it runs the same on any machine. You already know this from earlier projects.
- **Kubernetes (EKS):** the system that runs your containers, keeps them alive, scales them, and updates them. EKS is the managed version of Kubernetes on AWS.
- **Amazon RDS:** a managed database run by AWS, so you do not have to install and babysit a database server. ShopFlow uses PostgreSQL on RDS.
- **Amazon SQS:** a managed queue. One service drops a message in; another picks it up later. It lets services work without waiting on each other.
- **Amazon ECR:** AWS's storage for your container images (the things Kubernetes runs).
- **Nexus:** a store for build outputs (the compiled `.jar` files) and a cache for the open-source libraries your build downloads. This is the "artifact store".
- **Application Load Balancer (ALB) and Ingress:** the single front door that receives traffic from the internet and routes it to the right service, with HTTPS.
- **Amazon Cognito:** AWS's service for logging users in and proving who they are (authentication).
- **AWS Secrets Manager and IRSA:** Secrets Manager safely stores passwords and keys; IRSA is the secure way a service running in Kubernetes is allowed to read them, without any password being written in your code.
- **Prometheus and Grafana:** Prometheus collects numbers about your services (requests per second, errors, response time); Grafana draws them as live dashboards. This is **monitoring**.
- **Distributed tracing:** a way to follow one user request as it travels across several services, so you can see exactly where it slowed down or failed.
- **Terraform:** a tool that lets you describe your whole AWS setup as code, so you can create it, review it, and delete it reliably instead of clicking around the console. This is **infrastructure as code**.

You will not use all of these on day one. They arrive in order as you build, and each is introduced again at the moment you need it.

What each service does (the story of one order)

The clearest way to understand the system is to follow a single customer order through it. This is also the story you tell in an interview.

1. A customer opens the **Storefront** (the React web app) in their browser and logs in. **Cognito** checks their login and gives the browser a token that proves who they are.
2. The Storefront shows products. To get them, it asks the **Catalog service**, a Spring Boot backend that knows about products and inventory and reads them from the database.
3. The customer places an order. The Storefront sends that to the **Orders service**, another Spring Boot backend. The Orders service first calls the Catalog service to confirm the product exists and is in stock (this is one service talking to another, synchronously), then saves the order in the database.
4. After saving, the Orders service drops an "order placed" message onto the **SQS** queue and immediately tells the customer their order succeeded. It does not wait around.
5. Separately, the **Notifications worker** is watching that queue. It picks up the message and sends the confirmation. Because this happens through the queue, a slow confirmation never slows down the customer's checkout. This is asynchronous communication.

That single flow is the heart of the project. It shows a frontend calling a backend, a backend calling another backend, a backend saving to a database, and two backends working through a queue. Everything else in this document, the security, the monitoring, the pipeline, the AWS architecture, exists to build, protect, ship, and observe that flow.

The scenario (the "client brief")

A retail client needs a cloud-native order platform called ShopFlow. Customers browse a product catalog, place orders, and receive confirmation. The client wants it on AWS, resilient, observable, secure, and shipped through automated pipelines. This framing matters: students are solving a brief, not following a tutorial, which is exactly what they will describe in interviews.

The application and its services

A three-tier system made of independently deployable services:

Tier	Service	Responsibility	Stack
Frontend	Storefront	The web UI customers use	React single-page app
Backend	Catalog service	Products and inventory, REST API	Spring Boot (Java)
Backend	Orders service	Places orders, calls Catalog, persists orders, emits events	Spring Boot (Java)
Backend	Notifications worker	Consumes order events, sends confirmations	Spring Boot (Java)
Data	PostgreSQL	Stores catalog and orders (a schema per service)	Amazon RDS
Messaging	Order events	Decouples ordering from notifications	Amazon SQS

This small set deliberately exercises every communication pattern: frontend to backend, backend to backend (synchronous), and backend to backend (asynchronous), plus backend to database.

The backends are real Spring Boot applications, not toy servers: REST controllers, a service layer, JPA persistence, request validation, and Actuator endpoints for health and metrics. That is the "enterprise real backend" the project is built on.

The full AWS architecture

See [project6-aws-architecture.svg](#) for the diagram. In words:

- A **VPC** spanning two Availability Zones, with public subnets (load balancer, NAT) and private subnets (worker nodes, database). This is the foundation of a resilient design.
- An **Application Load Balancer** as the single entry point, provisioned by the AWS Load Balancer Controller from a Kubernetes Ingress, terminating **TLS** with an **ACM** certificate, with DNS in **Route 53**.
- An **EKS cluster** in the private subnets running the four services, each as a Kubernetes Deployment with its own Service for DNS-based discovery.
- **Amazon RDS for PostgreSQL** (Multi-AZ) in the private subnets, reachable only from the cluster.
- **Amazon SQS** for asynchronous order events between the Orders service and the Notifications worker.
- **Amazon ECR** for container images and **Nexus** as the build-artifact repository (the JARs and a dependency proxy).
- **AWS Secrets Manager** for database credentials and API secrets, delivered to pods through **IRSA** (IAM Roles for Service Accounts), so no secret is ever in code or a plain manifest.
- **Amazon Cognito** for user authentication; the frontend obtains a token, the backends validate it.
- Observability with **Prometheus and Grafana** for metrics, **Fluent Bit to CloudWatch** for logs, and **distributed tracing** across services.

How the services communicate

This is one of the core lessons, so it is explicit:

- **Frontend to backend:** the Storefront calls the backends through the Ingress over HTTPS, carrying a Cognito token.
- **Synchronous backend to backend:** the Orders service calls the Catalog service by its Kubernetes Service name (DNS-based service discovery, no hardcoded IPs), with timeouts and retries, and a fallback when the dependency is down.
- **Asynchronous backend to backend:** when an order is placed, the Orders service publishes an event to SQS; the Notifications worker consumes it independently. This teaches decoupling, why async matters, and eventual consistency.

Students learn the trade-offs directly: a synchronous call is simple but couples the services and fails together; an asynchronous event decouples them but is eventually consistent.

The CI/CD pipeline (reused and extended)

Each service is shipped by the gated pipeline from Project 5, one pipeline per service. On a push, the pipeline builds, tests, runs the SonarQube quality gate and Trivy scans, publishes the JAR to **Nexus** (the artifact store), pushes the image to **ECR**, and deploys to EKS. The frontend has its own simpler build-and-deploy pipeline. This is where the artifact store enters: build outputs are versioned and stored in Nexus, separate from the runtime images in ECR.

Security, built in (not bolted on)

- **Secrets:** AWS Secrets Manager plus IRSA. Pods assume an IAM role and read secrets at runtime; nothing sensitive is committed.
- **Authentication:** Amazon Cognito issues tokens; backends validate them and reject unauthenticated calls.
- **In transit:** TLS at the load balancer (ACM), and the database reached only privately.
- **Supply chain:** SonarQube static analysis, Trivy dependency and image scanning, all as pipeline gates (from Project 5).
- **In the cluster:** Kubernetes RBAC with least privilege, NetworkPolicies that allow only the pod-to-pod traffic that should exist, and hardened pods (run as non-root, read-only root filesystem, no unnecessary capabilities).
- **Infrastructure:** least-privilege IAM roles, private subnets for data and compute, security groups that expose only what is needed.

Observability

- **Metrics:** Spring Boot Actuator and Micrometer expose metrics; Prometheus scrapes them; Grafana dashboards show request rates, latency, errors, and resource use per service.
- **Logs:** Fluent Bit ships container logs to CloudWatch Logs for central search.

- **Tracing:** distributed tracing follows a single request across the frontend, Orders, and Catalog, so students can see exactly where time goes and where a call failed. This is the payoff lesson for microservice communication.
- **Alerting:** Prometheus Alertmanager raises alerts on error-rate or latency thresholds.
- **Health:** liveness and readiness probes on every service.

Testing strategy

A real testing pyramid, run in the pipeline:

- **Unit tests** with JUnit and Mockito for service logic.
- **Integration tests** with Spring Boot test support and Testcontainers, which spins up a real Postgres in a container so the data layer is tested against the real database.
- **Contract tests** between Orders and Catalog, so a change to one service that would break the other is caught before deploy.
- **Post-deploy smoke tests** that hit the live endpoints to confirm a release is healthy before it is considered done.

Infrastructure as Code

The entire AWS environment (VPC, subnets, EKS, RDS, SQS, ECR, IAM, Secrets Manager, Cognito) is defined in **Terraform**, so it is reproducible, reviewable, and can be torn down cleanly. This is the heart of "learning to architect": students declare the architecture, not click it.

The two phases

Phase 1: Build and run the system

Goal: a working multi-service application on AWS that you can use end to end.

- Provision the core infrastructure with Terraform (VPC, EKS, RDS, ECR).
- Build the three Spring Boot services and the React frontend.
- Implement the communication: Orders calls Catalog synchronously by service name, and emits an order event to SQS that the Notifications worker consumes.
- Persist data in RDS; manage configuration with ConfigMaps and database credentials with Secrets Manager.
- Expose the app through an ALB Ingress with TLS, and protect it with Cognito authentication.
- Deploy by hand with manifests (and Helm charts) and confirm the whole flow works.

What they learn: system design, three-tier architecture, synchronous and asynchronous microservice communication, frontend and backend and database integration, the AWS networking foundation, infrastructure as code, and baseline security.

Phase 2: Operate it as a platform

Goal: ship and run the system the way a real team does.

- Wire each service into the gated CI/CD pipeline from Project 5, publishing artifacts to Nexus and images to ECR.
- Add full observability: Prometheus and Grafana dashboards, centralized logs, distributed tracing, and alerts.
- Harden security: IRSA, NetworkPolicies, RBAC, pod security, and the supply-chain scans as gates.
- Add autoscaling (Horizontal Pod Autoscaler) so each tier scales under load.
- Add the full testing pyramid, including contract tests and post-deploy smoke tests.
- Package everything with Helm and, optionally, add canary releases.

What they learn: platform engineering, observability, security hardening, automated delivery of a whole system, scaling, and a professional testing strategy.

What this teaches, mapped to your goals

- **Platform engineering:** Kubernetes services and DNS, Ingress, ConfigMaps and Secrets, persistence, autoscaling, Helm, namespaces, RBAC, and running shared platform tooling (monitoring, artifact store).
- **System design:** three-tier architecture, stateless versus stateful, independent deployability, resilience across AZs, and the real trade-offs of microservices.
- **Microservice communication:** synchronous REST with service discovery, asynchronous messaging, timeouts, retries, and eventual consistency, made visible with tracing.
- **Frontend, backend, database:** a real UI calling real APIs backed by a managed relational database.
- **Security:** identity, secrets, TLS, supply-chain scanning, network and pod hardening, least privilege.
- **Architecting on AWS:** the whole environment designed and provisioned as code.

The hiring-manager narrative

After this project a student can say, truthfully:

"I worked on ShopFlow, a cloud-native order platform on AWS for a retail client. It was a microservices system on EKS: a React storefront, and Spring Boot services for catalog, orders, and notifications. The services talked over REST with Kubernetes service discovery, and orders were decoupled from notifications through SQS. Data was in RDS Postgres, the app was fronted by an ALB with TLS and Cognito authentication, and all secrets came from Secrets Manager through IRSA. The whole environment was defined in Terraform. Each service shipped through a Jenkins pipeline with SonarQube, Trivy scanning, tests including Testcontainers and contract tests, artifacts in Nexus and images in ECR. We ran Prometheus and Grafana with distributed tracing and alerting. I built X, debugged Y, and scaled Z."

To make that concrete, students keep a short STAR log as they go (Situation, Task, Action, Result) for three or four real moments: a tricky bug, a design decision and its trade-off, a security fix, and a performance or scaling change. Those become the stories they tell.

Tech stack summary

Concern	Choice
Frontend	React
Backend services	Spring Boot (Java)
Database	Amazon RDS for PostgreSQL
Async messaging	Amazon SQS
Orchestration	Amazon EKS
Ingress and TLS	AWS Load Balancer Controller, ACM, Route 53
Auth	Amazon Cognito
Image registry	Amazon ECR
Artifact store	Nexus
Secrets	AWS Secrets Manager via IRSA
CI/CD	Jenkins (the Project 5 gated pipeline)
Quality and security	SonarQube, Trivy
Monitoring	Prometheus, Grafana, Alertmanager
Logging	Fluent Bit to CloudWatch
Tracing	distributed tracing across services
Testing	JUnit, Mockito, Testcontainers, contract tests
Infrastructure as code	Terraform
Packaging	Helm

A necessary warning on cost and scope

This is a large, real architecture. EKS, RDS, an ALB, NAT gateways, and the supporting services cost real money, more than the earlier projects. It must be built deliberately and torn down with Terraform when not in use. Because of its size, the two-phase split is not optional; it is how the project stays learnable. Each phase is a real, runnable milestone on its own.